# 7 Boucles et organigrammes

Algèbre

**CE QUE VOUS SAUREZ FAIRE À LA FIN**

Exécuter et construire une boucle, produire un organigramme et justifier la terminaison d'un algorithme simple.

**JE RÉVISE AVANT DE COMMENCER**

Affectation, tests, reste de division euclidienne et calcul de sommes simples.

**TROIS MINUTES D'AUTOMATISMES**

**1** Calculer le reste de 17 par 5. Si $a = 3$, puis $a \leftarrow a + 2$, quelle est sa valeur ? Calculer $1 + 2 + 3 + 4$.

## 7.1 Exécuter sans sauter une instruction

**DÉFINITION 1 ▪**

Une affectation $a \leftarrow E$ remplace la valeur de $a$ par la valeur actuelle de $E$. Elle ne constitue pas une égalité à résoudre. Une boucle « pour » parcourt une liste d'indices ; une boucle « tant que » répète tant que sa condition est vraie. Le test de « tant que » précède chaque tour, y compris le premier.

**JE COMPRENDS SUR UN EXEMPLE 1 ▪ additionner les premiers entiers**

```
Lire un entier n ≥ 0
S ← 0
Pour k allant de 1 à n
    S ← S + k
Fin pour
Afficher S
```

Pour $n = 4$, les valeurs successives de $S$ sont $0, 1, 3, 6, 10$. Si $n = 0$, aucun tour n'est effectué et la somme vide vaut 0.

**PROPRIÉTÉ 1 ▪ invariant de la somme**

Après le tour d'indice $k$, $S = 1 + \cdots + k$.

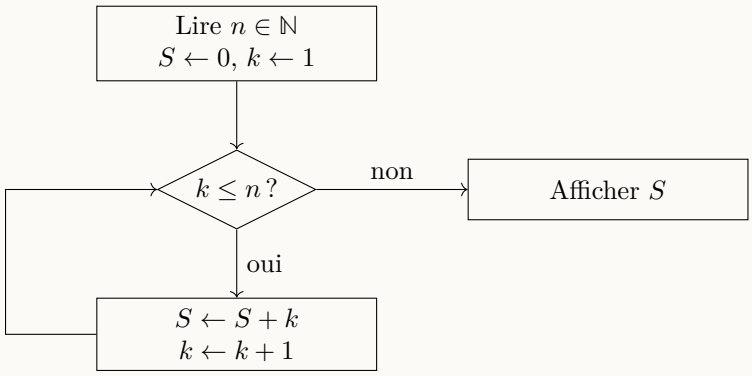
**DÉMONSTRATION**

Avant le premier tour, $S = 0$. Si la somme est correcte au tour précédent, l'ajout de $k$ donne celle des $k$ premiers entiers. Une récurrence justifie chaque tour ; après $n$ tours, le résultat annoncé est obtenu. ■

## 7.2 Un organigramme avec retour

L'organigramme suivant calcule la même somme. Le losange est un test ; les rectangles sont des affectations ; les flèches indiquent l'ordre d'exécution.

**JE VISUALISE ▪ le test décide de poursuivre**




Le compteur augmente d'une unité et finit par dépasser $n$. Le nombre de tours est fini.

## 7.3 Boucle conditionnelle et terminaison

**JE COMPRENDS SUR UN EXEMPLE 2 ▪ atteindre un seuil**

Avec $u \leftarrow 1$, $n \leftarrow 0$, tant que $u < 20$, faire $u \leftarrow 2u$ puis $n \leftarrow n + 1$. Les valeurs de $u$ sont $1, 2, 4, 8, 16, 32$ ; la sortie est $n = 5$. Après $n$ tours, $u = 2^n$. La suite des puissances dépasse 20 : l'arrêt est justifié.

**AVERTISSEMENT**

Si le corps ne modifie pas ce qui intervient dans le test, la boucle peut être infinie. Pour permuter deux valeurs, $a \leftarrow b$ puis $b \leftarrow a$ ne convient pas : la première valeur de $a$ a été perdue. Utiliser une variable temporaire.

**COMMENT ON S'Y PREND ▪ contrôler un algorithme**

Préciser les entrées autorisées ; dresser une table d'exécution ; expliquer ce qui reste vrai après chaque tour (invariant) ; expliquer pourquoi le test finit par être faux (terminaison) ; interpréter la sortie et les cas limites.

### Rédiger une table, une preuve et un test

**JE COMPRENDS SUR UN EXEMPLE 3 ▪ chaque ligne utilise les valeurs actuelles**

On initialise $a = 7, b = 3$, puis on exécute $a \leftarrow a - b$ et $b \leftarrow a + b$.

| État | $a$ | $b$ |
|---|---|---|
| Initial | 7 | 3 |
| Après la première affectation | 4 | 3 |
| Après la seconde | 4 | 7 |

Dans la seconde ligne de calcul, $a$ vaut déjà 4. Remplacer les deux variables simultanément par leurs anciennes valeurs décrirait un autre algorithme.

**COMMENT ON S'Y PREND ▪ séparer résultat correct et arrêt**

Un invariant explique ce que contiennent les variables si l'algorithme atteint une étape. Il ne garantit pas que l'algorithme s'arrête. Pour une boucle bornée, compter les tours ; pour une boucle conditionnelle, trouver une quantité qui finit par franchir le seuil. Tester aussi le cas où la condition est fausse dès le départ.

**JE COMPRENDS SUR UN EXEMPLE 4 ▪ dénombrer les termes déjà ajoutés**

Pour calculer $u_0 + \cdots + u_N$ quand $u_{n+1} = 2u_n + 1$, initialiser $u = u_0, S = u_0$. Au tour $k = 1, \dots, N$, mettre d'abord $u \leftarrow 2u + 1$, puis $S \leftarrow S + u$. Après le tour $k$, $u = u_k$ et $S = u_0 + \cdots + u_k$. En $N = 0$, il n'y a pas de tour et la sortie est déjà correcte. Inverser les deux mises à jour compterait deux fois $u_0$ et oublierait le dernier terme.

**AVERTISSEMENT ▪ atteindre une valeur ou s'en approcher**

Une suite peut tendre vers un seuil sans l'atteindre à un rang fini. Un test d'égalité exacte ne doit pas être remplacé tacitement par un test approché. Pour demander une précision, fixer une tolérance strictement positive dans l'énoncé.

## 7.4 Exercices et problèmes

**2** Suivre les valeurs Exécuter pour $n = 3$ : $u \leftarrow 2$ ; pour $k = 1$ à $n$, $u \leftarrow 3u - 1$. Donner les valeurs et la sortie.

**3** Somme de carrés Écrire un algorithme calculant $1^2 + \cdots + n^2$ pour $n \in \mathbb{N}$. Tester $n = 0$ et $n = 3$.

**4** Échanger Avec $a = 4, b = 7$, exécuter $a \leftarrow b$, $b \leftarrow a$. Réparer.

**5** Premier seuil Exécuter $u \leftarrow 3, n \leftarrow 0$ ; tant que $u \leq 100$, $u \leftarrow 2u$, $n \leftarrow n + 1$. Quelle différence ferait le test $u < 96$ ?

**6** Une boucle sans fin On initialise $u = 1$ et on répète $u \leftarrow (u + 1)/2$ tant que $u < 2$. La boucle s'arrête-t-elle en calcul exact ?

**7** Dessiner Transformer l'algorithme de la somme des carrés en organigramme avec un test avant la mise à jour. Indiquer les deux branches et l'actualisation de $k$.

**8** Une somme avec invariant Pour un entier d'entrée $n \geq 0$, on initialise $S = 0, k = 0$. Tant que $k < n$, on effectue, dans cet ordre, $k \leftarrow k + 1$ puis $S \leftarrow S + 2k$ ; on affiche $S$.

1. Dresser la table pour $n = 3$ et traiter séparément $n = 0$.
2. Prouver qu'après chaque tour $S = 2 + 4 + \cdots + 2k$ ; justifier l'arrêt et interpréter la sortie.
3. Que calculerait-on si l'on ajoutait $2k$ avant d'augmenter $k$ ? Tester $n = 3$.

**9** La limite ne suffit pas à arrêter En calcul exact, initialiser $u = 1, n = 0$ et répéter $u \leftarrow u/2$, $n \leftarrow n+1$ tant que $u > 0$. Cette boucle termine-t-elle ? Modifier seulement le seuil pour demander $u \leq 0{,}01$, puis calculer le premier rang d'arrêt et justifier sa minimalité.

**JE ME TESTE**

Je vérifie l'initialisation, le premier et le dernier tour, le compteur et le sens du test.